

课程编码	适用产品	产品版本	课程版本

作者/工号	时间	审核人/工号	新开发/优化
李春华/00891885	2024.07.22	新开发	

C++企业软件开发实践

实验3-2 面向函数式编程



前言

欢迎参加C++企业软件开发实践课程。本课程旨在通过完整开发案例，分享企业软件开发中的实践、经验和要求，帮助在校学生提升软件开发技能，养成良好的软件开发习惯。

实践课程共有4次实验，本课程为**实验3**，您将实践**面向函数式编程**，包括：

- **函数式编程、Lambda**：掌握C++新特性带来的代码简洁性收益，写出简洁优雅的代码
- **对象生命周期安全**：关注C++使用过程中编程陷阱可能导致的问题，对背后原理究根，学习C++特性整体体系

期待您在课程中的精彩表现！

目标

通过本课程的学习，您将能够：

- 掌握C++ Lambda表达式的优势
 - 代码简洁性：通过更紧凑的语法减少冗余代码，使程序更加简洁明了
- 深入理解C++编程陷阱
 - 识别和避免Lambda使用过程中可能出现的编程陷阱
 - 探究这些陷阱背后的原理，提升对C++特性整体体系的理解
- 提升代码安全性
 - 通过操作符()重载，将函数行为变成对象，增强代码的安全性和稳定性

这些学习目标旨在帮助您更好地掌握C++编程的最佳实践，建立高质量编程的**意识**，养成良好的开发**习惯**，提高代码质量和开发效率。

目录

1. 面向对象编程回顾

2. 项目实践

2.1 函数式编程

2.2 Lambda对象生命周期安全

2.3 操作符重载

3. 总结

面向对象编程问题分析

1. 继承实现多态

通过继承机制实现的多态性可能导致较高的耦合度。这是因为子类与父类之间存在直接的关系，子类必须实现或覆盖父类的所有接口

2. 接口实现的强制性

无论子类的具体需求如何，它都被迫实现父类定义的所有接口，这可能包括一些不必要或不相关的方法

3. 虚地址引发的问题

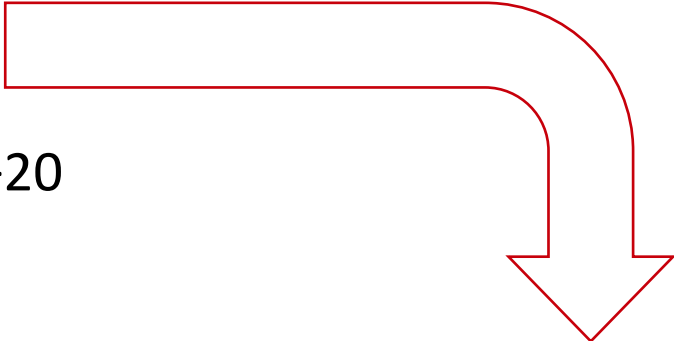
多态性通常涉及虚函数表，这可能导致在运行时定位问题的难度增加，特别是在出现错误或异常时

目录

- 1. 面向对象编程回顾
- 2. 项目实践
 - 2.1 函数式编程
 - 2.2 Lambda对象生命周期安全
 - 2.3 操作符重载
- 3. 总结

函数式编程

- 一种编程范式，强调对表达式的运算而非执行命令（声明式）
- 纯函数（pure function）
- 延迟计算（lazy evaluation）C++20



纯函数在C++中如何保障？

- 没有副作用：class中采用static，const关键字修饰
- 不变性Immutable

函数式编程实现：删除ICommand

删除ICommand

```
// class ICommand  
// {  
// public:  
//     virtual ~ICommand() = default;  
//     virtual void DoOperate(PoseHandler& poseHandler) const noexcept = 0;  
// };
```

函数式编程实现： MoveComand修改

```
...
#include <functional>
...
class MoveCommand final // : public ICommand
{
public:
    // void DoOperate(PoseHandler& poseHandler) const noexcept override
    // {
    //     if (poseHandler.IsFast()) {
    //         poseHandler.Move();
    //     }

    //     poseHandler.Move();
    // }
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.Move();
    };
};
```

函数式编程实现：TurnLeftCommand修改

```
class TurnLeftCommand final // : public ICommand
{
public:
    // void DoOperate(PoseHandler& poseHandler) const noexcept override
    // {
    //     if (poseHandler.IsFast()) {
    //         poseHandler.Move();
    //     }

    //     poseHandler.TurnLeft();
    // }
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.TurnLeft();
    };
};
```

函数式编程实现：TurnRightCommand修改

```
class TurnRightCommand final // : public ICommand
{
public:
    // void DoOperate(PoseHandler& poseHandler) const noexcept override
    // {
    //     if (poseHandler.IsFast()) {
    //         poseHandler.Move();
    //     }

    //     poseHandler.TurnRight();
    // }
    const std::function<void(PoseHandler& poseHandler)> operate = [] (PoseHandler& poseHandler)
noexcept {
    if (poseHandler.IsFast()) {
        poseHandler.Move();
    }

    poseHandler.TurnRight();
};
};
```

函数式编程实现：FastCommand修改

```
class FastCommand final // : public ICommand
{
public:
    // void DoOperate(PoseHandler& poseHandler) const noexcept override
    // {
    //     poseHandler.Fast();
    // }
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        poseHandler.Fast();
    };
};
```

函数式编程实现：表驱动修改

ExecutorImpl.cpp

```
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    std::unordered_map<char, std::function<void(PoseHandler & poseHandler)>> cmdMap;
    MoveCommand moveCommand;
    cmdMap.emplace('M', moveCommand.operate);

    TurnLeftCommand turnLeftCommand;
    cmdMap.emplace('L', turnLeftCommand.operate);

    TurnRightCommand turnRightCommand;
    cmdMap.emplace('R', turnRightCommand.operate);

    FastCommand fastCommand;
    cmdMap.emplace('F', fastCommand.operate);

    for (const auto cmd : commands) {
        const auto it = cmdMap.find(cmd);
        if (it != cmdMap.end()) {
            it->second(poseHandler);
        }
    }
}
```

编译运行验证后，
代码及时入库

目录

- 1. 面向对象编程回顾
- 2. 项目实践
 - 2.1 函数式编程
 - 2.2 Lambda对象生命周期安全
 - 2.3 操作符重载
- 3. 总结

函数式编程实现：代码问题分析

```
class MoveCommand final
{
public:
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        poseHandler.Move();
    };
};

class TurnLeftCommand final
{
public:
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        poseHandler.TurnLeft();
    };
};

class TurnRightCommand final
{
public:
    const std::function<void(PoseHandler& poseHandler)> operate = [](PoseHandler& poseHandler) noexcept {
        poseHandler.TurnRight();
    };
};
```

- 公开操作:

使用public关键字可能破坏类的封装性，因为它允许外部代码直接访问和修改类成员变量

- 封装性破坏:

缺乏足够的封装，业务逻辑直接暴露，从而带来安全隐患，同时也降低了模块的独立性和安全性

目录

1. 面向对象编程回顾
2. 项目实践
 - 2.1 函数式编程
 - 2.2 Lambda对象生命周期安全
 - 2.3 操作符重载
3. 总结

确保对象生命周期安全保证：MoveCommand操作符重载

```
class MoveCommand final
{
public:
    void operator()(PoseHandler& poseHandler) const noexcept
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.Move();
    }
};
```

确保对象生命周期安全保证：TurnLeftCommand操作符重载

```
class TurnLeftCommand final
{
public:
    void operator()(PoseHandler& poseHandler) const noexcept
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.TurnLeft();
    }
};
```

确保对象生命周期安全保证：TurnRightCommand操作符重载

```
class TurnRightCommand final
{
public:
    void operator()(PoseHandler& poseHandler) const noexcept
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.TurnRight();
    }
};
```

确保对象生命周期安全保证：FastCommand操作符重载

```
class FastCommand final
{
public:
    void operator()(PoseHandler& poseHandler) const noexcept
    {
        poseHandler.Fast();
    }
};
```

确保对象生命周期安全保证：表驱动修改

```
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    std::unordered_map<char, std::function<void(PoseHandler & poseHandler)>> cmdMap;
    cmdMap.emplace('M', MoveCommand());
    cmdMap.emplace('L', TurnLeftCommand());
    cmdMap.emplace('R', TurnRightCommand());
    cmdMap.emplace('F', FastCommand());

    for (const auto cmd : commands) {
        const auto it = cmdMap.find(cmd);
        if (it != cmdMap.end()) {
            it->second(poseHandler);
        }
    }
}
```

编译运行验证后，代码及时入库

简化表驱动代码：初始化语义应用

```
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    // std::unordered_map<char, std::function<void(PoseHandler & poseHandler)>> cmdMap;
    // cmdMap.emplace('M', MoveCommand());
    // cmdMap.emplace('L', TurnLeftCommand());
    // cmdMap.emplace('R', TurnRightCommand());
    // cmdMap.emplace('F', FastCommand());
    const std::unordered_map<char, std::function<void(PoseHandler & poseHandler)>> cmdMap {
        {'M', MoveCommand()},
        {'L', TurnLeftCommand()},
        {'R', TurnRightCommand()},
        {'F', FastCommand()},
    };

    for (const auto cmd : commands) {
        const auto it = cmdMap.find(cmd);
        if (it != cmdMap.end()) {
            it->second(poseHandler);
        }
    }
}
```

编译运行验证后，代码及时入库

目录

1. 面向对象编程回顾
2. 项目实践
 - 2.1 函数式编程
 - 2.2 Lambda对象生命周期安全
 - 2.3 操作符重载
3. 总结

本章总结

通过本课程的学习，您实践了：

- **函数式编程：**掌握C++新特性带来的代码简洁性收益，写出简洁优雅的代码

同时关注了使用过程中编程陷阱可能导致的问题，确保C++对象生命周期安全。

这些技能和知识将为您在未来的企业软件开发中打下坚实的基础，帮助您成为一名更加专业和高效的软件开发者。

感谢您参与本课程，期待您的未来的软件开发工作中取得更大的成就！

作业

- **状态变化简化：**通过状态变化，简化PoseHandler中的代码圈复杂度，实现优雅的代码结构
- **操作符()重载：**完成function操作符()的重载优化，提升代码效率和可读性
- **实现B指令需求：**包括测试用例

B&F指令正交分解的测试用例

状态 指令	B	F	BF	BB	FF
M	当前朝向E 执行BM X-1	当前朝向E 执行FM X+2	当前朝向E 执行BFM X-2	当前朝向N 执行BBM Y+1	当前朝向N 执行FFM Y+1
L	当前朝向E 执行BL 朝向S	当前朝向E 执行FL X+1, 朝向N	当前朝向E 执行BFL X-1, 朝向S	NA	NA
R	当前朝向E 执行BR 朝向N	当前朝向E 执行FR X+1, 朝向S	当前朝向E 执行BFR X-1, 朝向N	NA	NA

学习推荐

在线参考资料网站，涵盖了C++基本概念到高级特性、标准库函数、类和模板等各个方面

- 搜索功能：有强大的搜索功能，可以快速找到需要的函数、类或概念
- 示例代码：可以帮助理解如何使用C++特性
- 网址：<https://en.cppreference.com/w/>

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

**Copyright©2020 Huawei Technologies Co., Ltd.
All Rights Reserved.**

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

